

RehabBot — Répartition des Tâches par Membre

Projet : RehabBot — Assistant IA de Rééducation
Contexte : Module IA Bot — MSc MSI Promotion 2028 (EPITECH)
Durée : 7 jours · Groupe de 5 personnes · Soutenance : 20 minutes

Vue d'ensemble des rôles

Membre	Rôle	Bloc technique
Membre 1	Frontend / UX	Nuxt 3 / Vue.js (TypeScript)
Membre 2	Backend / Orchestration	Python / FastAPI
Membre 3	MCP Server / Scraping	FastMCP + BeautifulSoup / Zendriver
Membre 4	LLM / Prompts / Contexte	Ollama + Qwen3 1.7B / LLaMA 3.2
Membre 5	DevOps / Base de données / Documentation	PostgreSQL / SQLite + Docker

Membre 1 — Frontend / UX

Rôle principal : Développer l'interface utilisateur complète de RehabBot avec Nuxt 3 et Vue.js en TypeScript.

Tâches détaillées

Site vitrine

- Créer la page d'accueil présentant RehabBot (description, fonctionnalités, avertissement médical)
- Mettre en place la navigation et la structure de pages (layout Nuxt)
- Intégrer un design clair, accessible et responsive

Interface chatbot

- Développer le composant chatbot (zone de saisie, affichage des messages, historique)
- Implémenter la communication en temps réel via WebSocket avec le backend FastAPI
- Gérer l'affichage des états intermédiaires de l'agent : "analyse", "recherche en cours", "réponse"

Gestion des sessions côté client

- Stocker l'identifiant de session dans le navigateur (localStorage ou cookie)
- Afficher l'historique de conversation lié à la session active
- Gérer l'expiration de session (notification si session expirée après 3 semaines)

Variables d'environnement Frontend

- Configurer les variables `NEXT_PUBLIC_API_BASE` et `NEXT_PUBLIC_WS_BASE` dans `.env`
- Gérer les environnements dev / prod

Livrables attendus

- Pages : Home, Chat, À propos
- Composant `ChatBot.vue` fonctionnel avec WebSocket
- Indicateurs visuels d'état de l'agent

Membre 2 — Backend / Orchestration

Rôle principal : Développer l'API FastAPI qui orchestre l'ensemble du flux de données entre le frontend, le modèle LLM et le serveur MCP.

Tâches détaillées

Gestion des sessions et messages

- Créer les routes `/sessions` (créer, récupérer, supprimer)
- Créer les routes `/messages` (envoyer, récupérer l'historique)
- Implémenter l'expiration automatique des sessions après 21 jours
- Stocker les sessions et messages en base de données

Orchestration du flux IA (Agent Loop)

- Préparer le contexte de conversation (historique + nouveau message)
- Envoyer le contexte à Ollama via l'API locale (`OLLAMA_BASE_URL`)
- Détecter si le modèle émet un `tool_call` → appeler le serveur MCP
- Réinjecter le résultat du tool dans le contexte
- Récupérer la réponse finale du modèle et la renvoyer au frontend

Communication WebSocket

- Mettre en place un endpoint WebSocket pour le streaming de réponse en temps réel
- Envoyer des événements d'état intermédiaire (`status: "thinking"`, `status: "searching"`, etc.)

Intégration MCP

- Appeler les tools exposés par le serveur MCP (`search_exercises`, `search_sources`, etc.)
- Gérer les erreurs et timeouts lors des appels MCP

Livrables attendus

- Routes REST documentées (Swagger auto-généré par FastAPI)

- Agent loop fonctionnel avec gestion des tool_calls
- Logs des appels tools en base de données

Membre 3 — MCP Server / Scraping

Rôle principal : Développer le serveur MCP avec FastMCP, exposer les tools utilisables par le LLM, et implémenter le scraping de données publiques.

Tâches détaillées

Mise en place du serveur MCP

- Initialiser le serveur FastMCP sur le port 8001
- Définir les tools exposés avec leurs schémas d'entrée/sortie (JSON Schema)
- Tester la communication MCP avec le backend FastAPI

Développement des tools

Tool	Description	Implémentation
search_exercises	Recherche d'exercices par pathologie	Scraping de sites de kinésithérapie publics
search_sources	Recherche de sources fiables	Scraping HAS, Vidal, Ameli
get_rehab_advice	Conseils généraux par zone anatomique	Sites médicaux publics sans authentification
scrape_article	Extraction d'un article depuis une URL	URL fournie dynamiquement par le LLM

Règles de scraping à respecter

- Utiliser BeautifulSoup pour le parsing HTML statique et Zenderiver pour les pages dynamiques
- Respecter un délai de SCRAPING_RATE_LIMIT_SECONDS=2 entre les requêtes
- Ne scraper que des données publiques, sans authentification, sans CAPTCHA
- Ne pas contourner le fichier robots.txt des sites cibles
- Ne collecter aucune donnée personnelle

Nettoyage et structuration des données

- Extraire le texte pertinent (titre, corps, source, URL)
- Retourner des objets JSON structurés et exploitables par le backend
- Gérer les cas d'erreur (page indisponible, contenu vide, timeout)

Livrables attendus

- Serveur MCP démarré via `python app/server.py`
- 4 tools opérationnels avec retours JSON propres

- Fichier docs/architecture/mcp-tools.md décrivant chaque tool

Membre 4 — LLM / Prompts / Contexte

Rôle principal : Configurer le modèle local via Ollama, concevoir les prompts système, et optimiser la gestion du contexte conversationnel.

Tâches détaillées

Configuration du modèle local

- Installer et configurer Ollama en local (`ollama serve`)
- Télécharger et tester les modèles `qwen3:1.7b` et `llama3.2`
- Comparer les performances des deux modèles et recommander le plus adapté au projet
- Configurer la variable `OLLAMA_MODEL` dans `.env`

Conception des prompts système

- Rédiger le prompt système définissant le rôle de RehabBot (assistant de rééducation pédagogique)
- Intégrer l'avertissement médical dans le prompt pour que le modèle ne joue pas au médecin
- Définir les instructions de comportement : ton, format des réponses, gestion de l'incertitude

Gestion du contexte conversationnel

- Définir la stratégie de fenêtre de contexte (nombre de messages historiques à inclure)
- Gérer la troncature du contexte pour respecter la limite de tokens du modèle
- Structurer le contexte en sections claires : system prompt + historique + message utilisateur + résultat tool

Logique de décision `tool_call`

- Définir dans les prompts quand le modèle doit appeler un tool vs répondre directement
- Tester et itérer sur le format de sortie du `tool_call` pour qu'il soit bien parsé par le backend
- Implémenter des garde-fous (refus de répondre à des questions médicales critiques)

Tests et évaluation qualité

- Rédiger un ensemble de questions de test couvrant différentes pathologies
- Évaluer la qualité des réponses : pertinence, sécurité, clarté
- Documenter les limites connues du modèle

Livrables attendus

- Prompt système finalisé et versionné

- Recommandation de modèle documentée (Qwen3 vs LLaMA)
- Jeu de tests avec évaluation qualitative des réponses

Membre 5 — DevOps / Base de données / Documentation

Rôle principal : Assurer la stabilité de l'infrastructure, gérer la base de données, orchestrer les services Docker, et produire la documentation technique et de soutien.

Tâches détaillées

Base de données

- Concevoir le schéma de base de données : `tables sessions, messages, tool_logs`
- Implémenter les migrations avec `SQLModel` ou `Alembic`
- Tester la base en `SQLite` (dev) et `PostgreSQL` (prod)
- Implémenter la suppression automatique des sessions expirées (> 21 jours)

Docker et déploiement

- Écrire les `Dockerfile` pour le backend et le serveur MCP
- Configurer le `docker-compose.yml` pour orchestrer backend + MCP + base de données
- Documenter la procédure de démarrage rapide avec et sans Docker
- S'assurer que les variables d'environnement sont bien passées aux conteneurs

Scripts de démarrage

- Écrire les scripts dans `/scripts/` pour démarrer tous les services en une commande
- Prévoir un script de vérification de l'état de santé des services (`health check`)

Fichier `.env.example`

- Maintenir à jour le fichier `.env.example` avec toutes les variables nécessaires
- Documenter le rôle de chaque variable dans le README

Documentation technique

- Rédiger `docs/architecture/architecture-overview.md` (vue d'ensemble du système)
- Rédiger `docs/architecture/data-flow.md` (flux de données étape par étape)
- Maintenir le `README.md` principal à jour avec les instructions d'installation
- Préparer `docs/presentation/soutenance-notes.md` pour l'oral de 20 minutes

Livrables attendus

- `docker-compose.yml` fonctionnel
- Schéma de base de données documenté
- Documentation complète dans `/docs/`

- Notes de soutenance prêtes pour l'oral

Planning indicatif sur 7 jours

Jour	Activités clés
J1	Initialisation du repo, setup environnement, architecture validée en équipe
J2	Backend : routes sessions + messages · MCP : structure du serveur · Frontend : layout de base
J3	Backend : agent loop + intégration Ollama · LLM : premier prompt système
J4	MCP : tools de scraping opérationnels · Frontend : interface chatbot + WebSocket
J5	Intégration complète : frontend ↔ backend ↔ Ollama ↔ MCP
J6	Tests end-to-end, corrections de bugs, amélioration des prompts
J7	Finalisation documentation, préparation soutenance, démo finale

Règles de collaboration

- Git : une branche par fonctionnalité (feature/frontend-chatbot, feature/mcp-scraping, etc.)
- Code review : chaque PR doit être revue par au moins un autre membre avant merge
- Point quotidien : 15 minutes chaque matin pour synchroniser l'avancement
- Intégration : le Membre 5 est responsable de valider que tout fonctionne ensemble

RehabBot — Prototype pédagogique open-source. Ne pas utiliser en contexte médical réel.